CORA: Learning What to Change Through Executable Reasoning and Failure-Grounded Capability Construction

Technical report, November 2026. Not peer reviewed.

Chimdi Walter Ndubuisi* and Toni Kazic
University of Missouri *cnptp@missouri.edu

Abstract

When a reasoning system cannot solve a problem, more search may not help: its operations may be unable to express the solution. CORA (Constructive Operator Reasoning Architecture) treats failure as evidence. When search fails within a set budget, CORA uses the task's examples to describe what a missing step in a program must do (a typed hole). It tests possible operations from a predefined set and keeps one only if it passes checks. In a pre-registered experiment, we created 95 tasks, each missing one of 211 supplied operations. CORA solved 64 tasks in all three seeded runs. Using the examples directly solved 48, ordinary search solved 38, and using another task's information solved 16. Ordinary search used 3.2 times the CPU charged to construction. However, none of the 211 operations met the requirements of 425 failed benchmark tasks, so we developed a method for building operations that change individual objects. We then tested whether saved operations could help on other problems. The test covered 624 outputs from three ARC-style benchmarks, kept out of this study's development but used in earlier studies. Ordinary search produced 118 correct answers, reusing saved operations 227, and adding object-level construction 343; 286 of the 343 came from one-dimensional tasks, and on the 315 two-dimensional outputs the three methods produced 30, 11 and 57. We also rebuilt the approach for logic problems: it recovered 45 of 217 held-back logic functions, compared with 12 when the failure information was shuffled. On 84 generated tasks, a fixed rule for choosing what to try next solved 55, a preset order 29 and a random order 37; an advantage over the best order chosen after testing (44) was not established. CORA's Kaggle entry, which checks answers but does not build operations, scored 0.83% on 120 hidden ARC-AGI-2 tasks. These experiments show that a task's examples can help identify and reuse useful operations within a supplied set. Learning which change to make, and building operations beyond that set, remain open challenges.

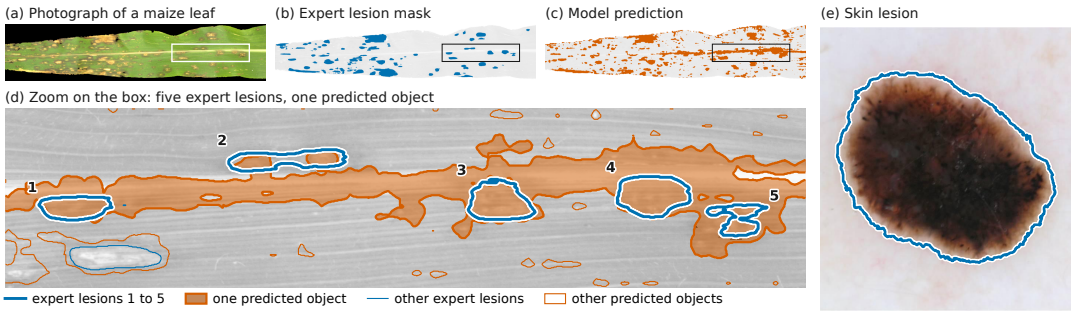


Figure 1: Lesions show why the representation comes first. (a) Maize leaf, (b) expert lesion mask, (c) model prediction; (d) one predicted object (orange) merges five expert lesions (blue). (e) A skin-cancer lesion and its expert boundary (ISIC Archive image ISIC_0000013 and its ISIC 2018 mask, both CC0; the ISIC record documents a histopathology-confirmed invasive melanoma, which the image itself does not establish). Motivation only, not CORA data and not a diagnosis: (a) to (d) show a validation leaf of our unpublished lesion study.

1 Introduction: why reasoning needs to change its own capabilities

A reasoning system explains observations with the operations it has: it searches over programs built from them and keeps one that reproduces what it has seen. When it fails, the cause can differ. The explanation

The extended version, with Appendices A to I, is in the public repository (Code and data).

may exist but be hard to find, the observations may be described with the wrong objects, or an operation the explanation needs may be missing; each cause calls for a different change. Our broader question, which grew out of our research discussions, is whether a system can learn from a few observations, notice when its rules fall short, work out what must change, change it, and carry the same way of reasoning to a new domain.

Lesions show why the representation comes first (Figure 1). For object-level reasoning about lesions, the chosen boundaries constrain which rules can work. On the maize leaf, one predicted object merges five expert lesions; rules using that object inherit the merge unless the representation changes. Our unpublished leaf-lesion work, which does not use CORA, pointed the same way: what had to change was seeding, where objects start, not geometry [49, 50]. When a separate study ran CORA's image pipeline on our leaves, with every decision frozen before any evaluation mask was read, a segmenter trained on cell images found almost none of the lesions on 16 evaluation leaves, CORA's checks refused every edit, and a plain search's mostly helpful edits (103 of 120) left F1, the lesion-detection score, near zero: the tested local patches could not rescue a wrong representation.

Searching longer and changing the language are different responses. Refinement loops, the defining theme of ARC Prize 2025 [18], improve a program or a network within a supplied language. Longer search reaches behaviour that deeper compositions express; behaviour outside the language's expressible set needs a new operation, and behaviour reachable only through too deep a composition can benefit from one: *capability construction*.

When reasoning fails, the failure is evidence; CORA uses that evidence in two distinct ways. In the ARC-style construction experiments, a failed bounded search triggers construction, and the task's demonstrations specify what the missing operation must do. In the logic-function experiments, the failure description itself carries information used to choose the revision: shuffling it removed most of the gain. In the second selection test, the deciding fact is a property of the demonstrations stored in CORA's failure record, whether every demonstration keeps its grid size; it does not come from the failed search. In every case a proposed change is checked against the observations before it is kept.

CORA (Constructive Operator Reasoning Architecture) turns this into one procedure: observe, represent the observations as objects and relations, search for an executable program, and answer only with a checked program; when bounded search fails, work out a *typed hole*, the input and output pairs a missing operation must produce, test candidates from a declared space against it, and keep one only if it passes explicit admission checks (Figure 3 follows one recorded task). Three requirements organise the evidence: *capability availability* (can the language express the explanation?), *capability discovery* (can the system find it within its budget?) and *capability selection* (can evidence decide which representation, operation or strategy to use?). Construction expanded budgeted reach and improved discovery inside a supplied space; selection is the open frontier. The same procedure was rebuilt for other domains through *adapters*, which translate a domain's observations into the loop's entities and operations: the procedure transfers, rebuilt per domain; a domain's own evidence beat shuffled evidence in two of five new domains, and learned grammar transfer across distant domains is not shown. The box separates what was scored from these research studies.

Scored entries. Kaggle submission 57902131 runs CORA alone, on CPUs, with no neural model. It submits a program-derived answer only after checking it, and otherwise a copy of the query input, its fallback for an internal abstention (on the public evaluation set its only 3 correct outputs were its 3 answers). Kaggle public-leaderboard score on 120 hidden ARC-AGI-2 tasks: **0.83%**. Submission 57867631, the neural comparison entry, runs our re-implementation of NVARC's published method on NVARC's checkpoint [65], with a checked CORA second attempt: **24.03%**, 3.6 points below NVARC's own notebook, which we reran as a reference (submission 57160429, not ours: 27.64%). CORA's second attempt gained 0 and lost 0 answers on the public evaluation set, and its effect on the hidden tasks is not measured. The full research paper [51] holds every study, control and cost.

Scope. The scored entries run search and checking only (an earlier entry with failure-driven construction, 57354496, scored 0.00%); this paper's construction, reuse, the learned order, adapters and biology code ran in separate studies. We supplied the constructor's space of operations, and fixed rules decide when to construct. The generated tasks test the mechanism; they are not ARC accuracy. Held-back halves were held out from each study's development only: every task had been scored once on 2 November 2026 by our submitted entries and once by a parallel study's evaluation. Boolean gene-network observations are simulated from published models, so those results are reconstruction, not discovery; the single-cell, microscopy and spatial studies use measured data and support scoped diagnoses or adapter tests.

Contributions. The ingredients have precedents, and Appendix H states each prior system and the exact difference. Our contributions are implemented designs, controlled comparisons and identified limits:

1. *What a build reads decides what gets built.* Bias shift and predicate invention, also from failures, add to a learner's vocabulary, and Popper and Neo learn failure-based constraints [20, 21, 24, 48, 67]; counterexample-guided synthesis refines candidates within a fixed grammar, abstraction refinement refines an abstraction from a spurious counterexample, and witness functions derive what a missing piece must do within a fixed language [19, 56, 64]. Our pre-registered controls change only the evidence a build reads: over the same 211 operations, a task's own typed holes solved 64 of 95 generated tasks, the operations tried blind 32 (by three-repeat majority) and another task's holes 16; in logic, 45 of 217 held functions against 12 with the failure description shuffled.
2. *Availability and discovery, measured on the same tasks.* DreamCoder, LILO and Stitch learn abstractions from discovered programs [9, 23, 27]. Of 624 previously used held outputs, the failure-kept library got 227 (216 on 1D-ARC; known beforehand), same-size compression of solved programs 28, random libraries 6, with unequal build CPU (not a general method comparison).
3. *When choosing the change can pay*, from two pre-registered revisions of a selection test, beside learned search guidance, which in DeepCoder and CrossBeam beat its baselines, and metareasoning [6, 58, 61].
4. *One loop design, rebuilt by adapters* for five new domains, each frozen before its held run, every held result reported with its scope, a null and a failed test among them; FlashFill serves one domain [28].

Sections 2 and 3 give the mechanism and its limits in principle, Sections 4 to 7 test construction, reuse, selection and transfer, and Sections 8 and 9 turn the failures into the next experiments.

2 From failure to capability construction

Why ARC-AGI-2. A few input-output examples constrain a hidden transformation, and candidate executable programs can be checked against them without a human judge [11, 17]; each failed program leaves a record of what went wrong, two attempts are allowed per query output, and the scoring tasks are hidden [18, 35]. The Boolean gene-network studies use simulated examples, which likewise constrain a hidden rule.

Terms. A *task* gives *demonstrations*, pairs of input and output grids, and *queries*, test inputs whose *query outputs* must be produced; a system *answers* a query output by giving a grid, *correctly* if an attempt matches exactly, and *solves* a task by answering its query outputs correctly. A *program* is a short sequence of typed operations; a *language* is the operations and the rules for combining them; the *starting language* is the language before any operation is added, and *static search* searches it with no new operation. In the scored entry, *checking* runs a program on every demonstration and rebuilds it without each one; *abstention* is declining to answer. *Held out* means kept away from a study's development; *exposed* means used before in this project, so held-out exposed tasks are never unseen. *Pre-registered* means frozen before scoring, and *audited*

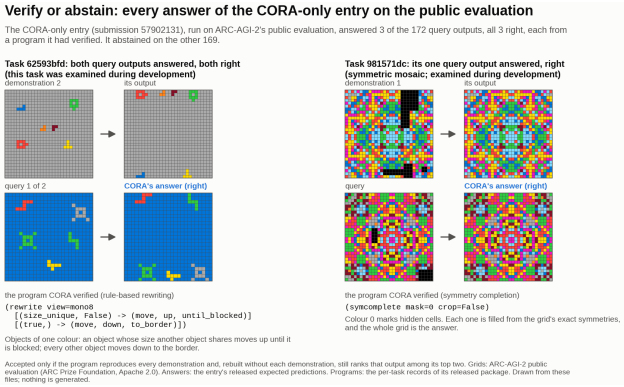


Figure 2: An actual executable ARC explanation: every answer of the CORA-only entry on the public evaluation set, drawn from the task file and the entry's recorded programs (an actual CORA run). It answered 3 of 172 query outputs, all 3 correct, each from a program that reproduces every demonstration and, rebuilt without each demonstration, still ranks that demonstration's output among its top two; it abstained on the rest. Both tasks were inspected during development, so this shows how CORA explains a task, not an unseen accuracy.

means checked after scoring, from saved records, by a separate review within this project, not by external replication. Each study states its *counted-work* unit (candidate applications, called *counted operations*, or primitive executions): machine-independent, and not interchangeable across studies.

The scored entry: answer only with a checked program. CORA's grid adapter reads each grid as objects, connected regions cut in one of several ways (*views*), with colour, size, shape, position and relations. The CORA-only entry runs two CORA solvers in turn. The main one, CORA-CAST, builds programs with symmetry completion, guarded rewriting (short first-match lists of "if an object has this property, do this action") and typed search pruned by sound contracts. It answers only if a program reproduces every demonstration and, rebuilt with each demonstration left out, still ranks that demonstration's output among its top two, and never from a single demonstration; otherwise it abstains, within 45 CPU seconds and 90 wall seconds per task. A second reviewed CORA solver from the companion study, a parallel line of our experiments, adds stricter leave-one-out checks. Run once, unchanged and untuned, the entry's code was correct on 81.5% to 95.8% of the query outputs it answered on three related benchmarks, while answering only 11% to 50% of them, and with another task's demonstrations precision fell to at most 2.4% (exposed; pre-registered): on exposed sets coverage looks like the main limit, and how the official 0.83% splits into precision and coverage on the hidden tasks is not observed.

Construction: from a failed search to a kept operation. The scored entry stops at checking. When search fails, a separate research prototype tries to construct the missing operation. A program sketch puts the hole alone, or before or after one invertible starting-language operation; undoing that operation (if any) on the demonstrations gives the exact input and output pairs the missing piece must produce: the failure triggers the step, and the demonstrations supply its content. In the generated-task study all 211 declared whole-grid operations are checked on these pairs, in about 0.02 CPU-seconds, and only matches enter a search; the held-half study's constructor is object-level instead (Section 5). A candidate is kept only if a bounded search in the starting language does not reproduce it, it fits the task, it breaks no solved task and it fits another unsolved task; kept pieces join the language. This is selection within a known space, not open invention: we supplied the operations and the diagnosis follows fixed rules. Learning enters only through kept operations and a learned search order, the direction the organisers' 2024 report expected for guiding program search [16].

A worked example. Figure 3 follows one of the 95 generated tasks. Its four demonstrations each shrink a grid, and the task was generated using an operation omitted from the starting bank. Search still found no fit at four times the budget; with diagnosis unresolved, the controller chose construction. The top sketch is the hole alone, so the hole equals the raw demonstrations; 4 of the 211 passed it, and the one sent to the checks, a row-and-column selection, passed them all. Seeds 1 and 2 admitted that selection and seed 0 a sibling variant; each solved the task, and saved results show five later tasks solved using the retained variant.

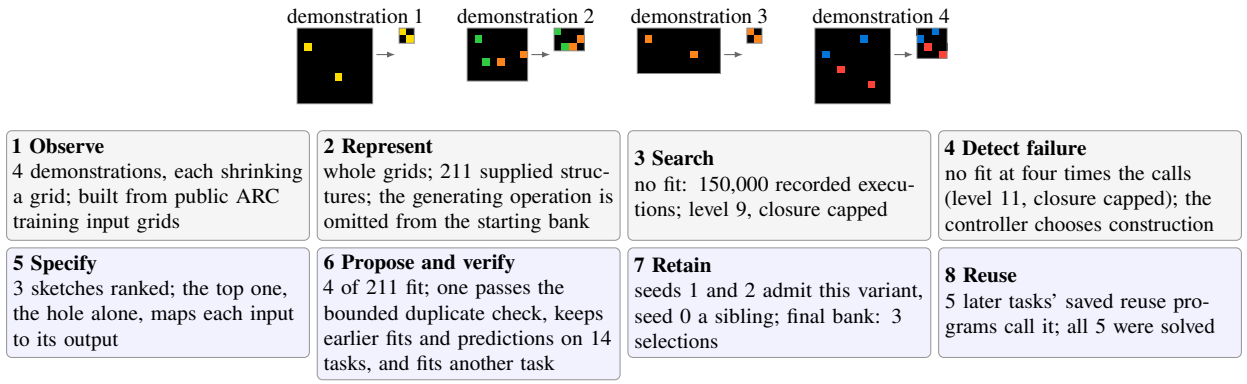


Figure 3: CORA's mechanism on one generated task of the construction study, step by step from its records (an actual run of the research prototype; the scored entry stops after checking). Top: the four demonstrations (the query is not drawn); bottom: the recorded sequence. Seed 0 admitted a sibling variant, which the committed language does not hold. The top sketch is the hole alone, so here the typed hole equals the raw demonstrations, which also solved the task; the same 211 operations tried blind solved it in two of three repeats; another task's holes and static search failed (no fit after 379 CPU-seconds and 22.9 million recorded executions, level 14, closure capped). Every search had a cap, so this does not show the program inexpressible. The five reuse outcomes are the original run's saved results, not recomputed and not proof that the operation was necessary. For a task where the task's own typed hole succeeded and the other arms failed, see Section 4 (our key td0c3d452).

3 Theoretical limits of search, construction and selection

The worked example raises three questions. Can demonstrations identify the intended program? Not necessarily: several programs can reproduce the same observations and differ elsewhere. Can better selection make up for an unavailable operation? No: a selector cannot choose an answer absent from its candidates. Can enlarging the language create new problems? Yes: added fitting programs can increase ambiguity under some acceptance rules. Four elementary conditional results, proved in Appendix B under their stated assumptions, make these answers exact; Table 4 separates empirical illustrations from the untested prediction, the results organise the measurements rather than predict them, and no priority is claimed. Let D be the demonstrations, x^* a query, $\mathcal{L}$ a language, $V_{\mathcal{L}}(D)$ the programs of $\mathcal{L}$ that fit D (reproduce every demonstration), and $A(D, x^*)$ their predictions at x^*.

Proposition 1 (Many explanations)**.** *If some program of $\mathcal{L}$ fits D, $\mathcal{L}$ can return any allowed output on an exact match to x^* and otherwise run any program of $\mathcal{L}$, and x^* differs from every demonstration input, then every allowed output is predicted at x^* by some program in $V_{\mathcal{L}}(D)$ [46].*

Proposition 2 (Build before you rank)**.** *If attempts are drawn from the retained predictions within budget b, no selection rule, however learned, raises accuracy above the share of tasks whose answer is retained, and that share cannot exceed the share of tasks whose answer some program of $\mathcal{L}$ that fits D predicts at x^*.*

Proposition 3 (Bigger language, more ambiguity)**.** *Suppose an answer is given only when every fitting program returns the same output at x^*, a program that returns nothing counting as disagreement, and $\mathcal{L} \subseteq \mathcal{L}'$. On every task that $\mathcal{L}$ already fits, growing to $\mathcal{L}'$ never adds or changes an answer; it turns an answer into an abstention exactly when $\mathcal{L}'$ adds a fitting program that does not return that output at x^*.*

Proposition 4 (A missing input blocks exact recovery)**.** *Say that a rule depends on an input v when some two inputs that differ only in v receive different values. If a rule depends on v and every candidate gives equal values to any two inputs that differ only in v, no candidate equals the rule, however good the search; and for queries fixed in advance, or chosen by a fixed deterministic policy from earlier binary responses, and with no target-dependent side information, m binary responses can single out at most 2^m rules [1, 4].*

What they mean for CORA. Passing the examples shows consistency, not the intended rule (Proposition 1), which is why CORA's answer rule also rebuilds each program without each demonstration: of 4,832 constructed rules proposed by two or more demonstrations and fitting all, 562 were wrong on the query (57.9% on ConceptARC; post hoc, development data). A selector cannot return what the language cannot

express or search cannot reach (Proposition 2): on the 41 query outputs that only the neural comparison entry answered correctly, ten times CORA's search budget found no fitting program, so no ranking of CORA's candidates could have answered them. Growth has a cost under all-agree acceptance (Proposition 3, not yet tested). A missing input blocks every candidate (Proposition 4), the gene-network form of a missing operation (Figure 5b), and the bound is why choosing observations matters. These mathematical statements, under their assumptions, say when construction and selection are necessary; the experiments below test whether CORA's implementation uses that opportunity within a budget, and the propositions prove no empirical advantage.

4 Does construction improve capability discovery?

Question and design. If the language lacks an operation, can task-specific evidence guide its construction better than search or blind trial? A frozen study built 95 tasks from public ARC training input grids in three worlds whose languages lack 11 of the 211 operations (scalings, crops, symmetries, colour swap; pre-registered; audited; three seeded repeats). Each control changes one ingredient (Table 13, Appendix I).

Result. Construction from each task's own typed holes recovered 64 in all three seeded repeats (60 by admission, 4 by a fitting fallback), against 38 for static search, which used 8.47 measured CPU-hours in one run, while construction was charged 2.61 CPU-hours for its three repeats under the protocol's complete-cost rule, which charges each cached admission check at its first computation (measured growth about 0.31 CPU-hours plus 902 diagnosis CPU-seconds): 31 tasks gained, 5 lost, a difference of 27.4 points (95% interval over eight family clusters, +5.1 to +40.6). The raw demonstrations as the hole recovered 48 (16 gains for the typed holes, 0 losses), another task's holes 16, all in the one world whose removed operations compose (0 of 59 in the other two), and the same 211 operations tried blind 32 (3 in all repeats, 74 in any; Figure 4a).

Principal finding. With the supplied candidate operations and the checks fixed, holes from each task's own demonstrations recovered more generated tasks than tested alternatives (64 against 48 for raw demonstrations and 16 for another task's holes): within these generated tasks, the information read caused the gain.

Depth and limits. Post hoc, the 31 tasks that only construction solved split into the two requirements (Figure 4b): 13, the worked example among them, have no equivalent starting-language program at any depth searched (not proof that none exists deeper), 17 have one 1 to 4 cost levels deeper, an estimated 1 to 135 times static search's budget (median about 5), and 1 was a static-search process failure. On one of the 17 (our key td0c3d452), the winning recorded sketch first mirrors the inputs, then fits the colour exchange, in all three repeats; raw demonstrations, a budgeted blind trial and another task's holes failed (saved outcomes; an illustration, not an accuracy result). Leaving out either of two worlds gives intervals that include zero, and every removed operation belongs to the space by design: selection within a known space, not open invention. Is a constructed operation useful beyond its own task? Within this study, operations reused on other families' tasks did worse than static search (23 against 28 of 60; Holm $p = 0.47$).

5 Can constructed operations be reused?

Question and design. Learning a useful operation differs from learning one that generalises. Object-level operations built on the source halves of ConceptARC [47], Mini-ARC [34] and 1D-ARC [69] ran once on the held halves, 624 query outputs of 464 tasks, every one scored once before by the submitted entries (exposed; pre-registered; audited). The held halves come from the same task families as the source halves, and the object-level constructor fits one or two first-match rules, each a supplied predicate (among them 24 selection features) with a supplied object edit, over 12 views (Appendix I).

Result. Reuse, the pre-registered primary, was correct on 227 against 118 for static search in the starting language with the same CPU allowance (+109, 95% interval 7 to 204), all of the gain on 1D-ARC; on the two two-dimensional benchmarks reuse lost (11 against 30 of 315). Object-level construction from each task's own demonstrations, where reuse failed (a pre-registered secondary), was correct on 343 of 624 against the same 118 (it answered 377; interval for the difference 150 to 298), and on 57 against 30 of 315 in two dimensions.

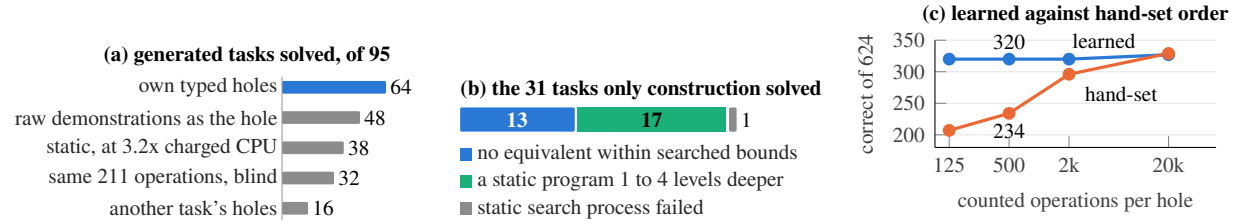


Figure 4: How well construction and search guidance work: the central controlled evidence, from actual runs of the research prototype (Figure 3 shows the mechanism). (a) Generated tasks solved, of 95, each lacking one operation (pre-registered; audited), counted in at least two of three seeded repeats; static search used 3.2 times the CPU charged to construction (measured against charged); the blind trial of the same 211 operations solved 3 in all three repeats, and the other four counts are the same in all three. (b) Post hoc: the 31 tasks that only construction solved; "no equivalent within searched bounds" is not proof that none exists deeper. Every removed operation belongs to the space by design (for 7 of the 11 families the space was fixed before the operation was chosen, and the other 4 operations predate it), and the blind control searches the same space: selection within a known space, not open invention. (c) Correct query outputs of 624 held, exposed outputs at four caps on counted operations per hole; the hand-set order first reaches 320 at 20,000, 40 times the test cap of 500 (a sampled per-hole cap, not total compute and not a proven minimum).

Static search given the same expanded operation bank has not been run. A library of whole programs beat the starting language (37 against 6) but lost to static search given the library's whole cost (37 against 56 of 624); that tests our library, not DreamCoder or LILO [23, 27]. On the 120 public ARC-AGI-2 evaluation tasks, construction correctly answered 2 of 172 query outputs, both on the left task of Figure 2, which the CORA-only entry already answers; every other condition solved none (95% interval for the difference from static search, 0 to 6 outputs): a descriptive, exposed example, not evidence that the operation was necessary; reach is the limit there.

What it means. Reuse can help only where operations were kept: it gained on 1D-ARC (19 kept) and lost in two dimensions (2 kept, none on Mini-ARC). When should the system reuse an operation, search longer, change its representation or construct another one?

6 When does evidence help select a revision?

Two activities must be kept apart: *search guidance* decides where to search within an available operation space, and *revision selection* decides which kind of change to attempt. The left task of Figure 2 shows why selection is hard: five earlier versions of our system failed on it, with no fitting program, for four different reasons, each needing a different change (Appendix I; inspected during development: an illustration).

Search guidance helps when the budget binds (exposed; pre-registered; audited). A small guide trained on earlier successes and failures replaced the constructor's hand-set order (Figure 4c): at 500 counted operations per hole it was correct on 320 of the 624 held query outputs against 234 (search CPU 115 against 106 seconds); at a per-hole cap 40 times larger, one of four sampled caps and not a measure of total compute, the counts were close (327 against 329; inconclusive). A frequency table recovered 318 and a linear model 323, against 320 for the learned guide, so in this study the learned guide shows no advantage over simpler rankers. A later guide reading each task's grid pairs and the constructor's own analysis of its search blocks, frozen and reviewed before any held task existed, solved 1,083 of 1,128 held tasks generated from ARC-AGI-2 public training input grids at 125 counted operations each, against 953 for the hand-set order, 881 for a frequency table and 1,018 for a linear model (pre-registered; audited): search efficiency at a binding budget, not new reach; a guide reading only the constructor's own block features solved 1,076, not significantly fewer.

Revision selection (generated; pre-registered; scored once; audited). Two revisions of one test varied only the order of the same changes under the same resource caps and work meter; an *oracle* router is told each task's injected cause. In revision 1, typed-hole construction rejects an inapplicable hole after about 900 calls, so trying it first costs almost nothing, and the best fixed order matched the oracle (30 of 50 each): choosing added nothing. Revision 2 added a change that cannot check cheaply whether it applies, a search over alternative object views. In a pre-registered test on 84 controlled generated tasks (transformation tasks

built from ARC training input grids, not ARC puzzles), each with one injected cause of failure, a fixed router that reads one fact from CORA's failure record, whether every demonstration keeps its grid size, solved 55 tasks at the pre-declared binding budget of 3,000,000 primitive executions, against 29 for the static full bank in its pre-registered order (26 tasks against 0; exact McNemar p = 3.0e-8; 95% interval over resampled task families, +11.8 to +55.2 points). The whole gain came from the 28 wrong-object-view tasks, and at every budget the router solved exactly the oracle's tasks.

Its limits. Against the best fixed order chosen after seeing the held results (construction, then the view search, then deeper search: 44 of 84), the router's advantage is not established: 11 tasks against 0, with an interval from 0.0 to +26.1 points. The size fact separates the causes because of how the tasks were generated, and it comes from the demonstrations rather than from the failed search. The pre-registered static order scored below a random order on the held mix (29 against 37), and two of the three held causes reuse revision 1's held families with fresh input grids. A learned Boolean selection study's pre-declared gate stopped its held study, so no held test ran (Appendix D).

What it means. Where a change cannot cheaply check whether it applies, no fixed order of the bank reached the oracle (best 44 against 55 of 84), and the pre-registered router did, with no established advantage over that best order: choosing the next revision can pay when demonstration evidence separates the injected causes and an unsuitable change is costly. This is not revision selection on natural ARC tasks, where all CORA programs together solve 2 of the 120 public tasks, too few to compare methods. What carries over to other domains?

7 Does the reasoning procedure work beyond ARC?

What transfers. Transferability here means that the reasoning pipeline carries over (build an executable rule, check it, diagnose failures, revise or ask for evidence, abstain when unsure), not that puzzle rules apply to genes. The loop design is reimplemented through domain-specific adapters, using supplied building blocks and checks. An observation is a grid pair or a simulated network state paired with one gene's next value. Entities are objects or genes, programs are grid transformations or gene update rules, and inferred inputs remain candidates. Checks compare outputs. Spatial adjacency is not regulation: the correspondence holds only through an adapter that declares its meaning. Evidence-guided extension within supplied spaces helped in some tests: reading the domain's own evidence beat a shuffled or borrowed one on generated grid tasks, in logic and in signalling, and is not established in spreadsheets, chemistry or ARC-AGI-3 (Table 12); on measured imaging, the grammar built from failures was empty and no robust benefit emerged. Learned content transfers within 1D-ARC (Section 5) but not yet between distant domains: an operator order counted on ARC training tasks did not help gene rules (177 against 194 for a hand-set simplicity order). The scored CORA-only code ran unchanged on 44 published gene-network models but recovered no rule, so the gene results come from a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code.

Controlled reconstruction: logic functions (held; pre-registered; audited). Each new-domain test rebuilt the loop through an adapter (building blocks, a checker and a failure description; each study's public adapter card lists what was reused). On 217 held real benchmark logic functions (the Espresso example files, which include the MCNC benchmarks; 6 to 12 inputs), recovered exactly from a random 25% of each truth table, the loop recovered 45, against 7 for the base language of two-input gates and 12 for the same loop with a shuffled failure description (34 to 1 on discordant functions; file-clustered 95% interval for the difference 0.075 to 0.241); CART, a descriptive reference, recovered 39. The shuffled control keeps every program the base language finds, so it tests whether the failure description itself carries the gain. The gain spread over 13 of the 35 files, 153 of the 217 were recovered by none of the six methods, and every recovery reconstructs a known function. Choosing three quarters of the probes where the surviving candidate programs disagreed most raised exact recoveries from 45 to 63 of 217 at the same probe budget.

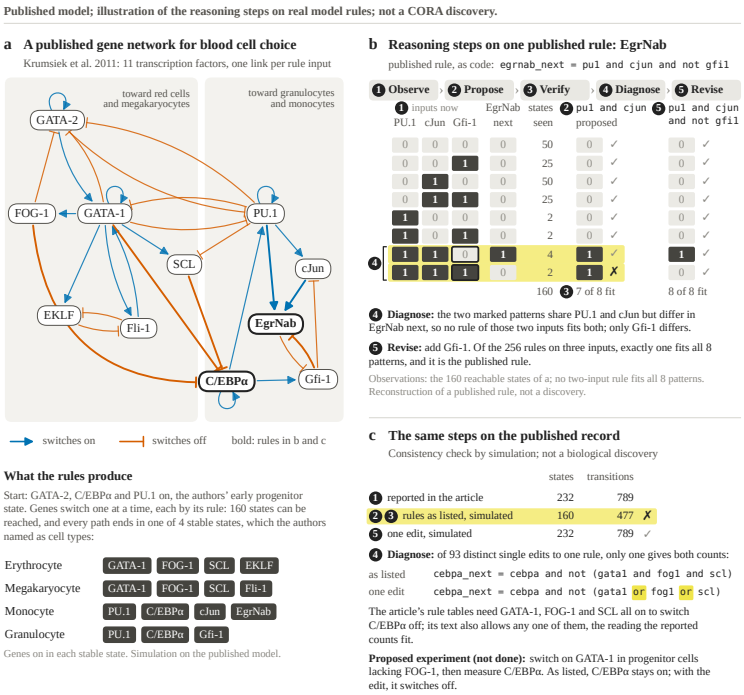


Figure 5: The reasoning procedure on a real rule of a published blood-cell gene network: a reconstruction from a published model, not a CORA run and not a discovery. (a) The 11 transcription factors of Krumsiek et al. [38] and their update rules, which from the progenitor state end in four stable states, the blood cell types. (b) A simpler rule is insufficient: the best two-input rule for EgrNab fails on one observed pattern; adding Gfi-1 to the illustrated candidate inputs leaves one of 256 three-input rules, the published one. (c) One edit to a listed rule matches the article's state counts (a consistency check by simulation), and a proposed experiment, not done, would separate the two readings. CC BY article.

Biological reconstruction: published gene networks (simulated; reused cohort). A published gene-network model gives each on-or-off gene an update rule over input genes, its *regulators* in that model; an observation pairs a simulated network state with that gene's next value, and exact recovery means a rule that agrees with the published one on every possible input state. On 595 rules from 75 models in 50 publication lineages, preferring candidate regulators inferred from each rule's ten simulated observations (kept apart from the documented ones), the biology code recovered 101 rules exactly, against 66 for each of three controls (two with no regulator preference, one with shuffled regulators), in a replication frozen before its single run; all 101 are one-input rules, more wrong rules were accepted (124 against 112), no classical support-inference method ran, and the models had been parsed before. Twenty observations chosen where fitting rules disagree gave 320 exact rules against 291 for random ones, but fewer correct held inputs (1,586 against 1,747 of 2,380). Conditional construction, which builds a rule over more inputs from two simpler rules, recovered 433 rules exactly at 40 observations, against 321 for the complete two-input rule language; all 112 additional rules have three or more inputs, so this shows executable coverage from a larger supplied language, the biology counterpart of building a missing operation, but a classical learner given the same observations also reached 433, and the run does not test whether a failure triggered the gain. Figure 5 shows the reasoning on one published rule; CORA's biology reaches computational validity only (Appendix E).

Other computational domains (held; pre-registered; audited). The other held tests are in Table 12 (Appendix I), with their controls and limits. Spreadsheets were a null (22 against 17 and 17 of 53). In signalling, 863 of 1,000 held conditions were predicted exactly against 805 with a shuffled description; 44 of 58 gains came from mek12 to erk12 (one of six edges); elsewhere 2.0 points, not significant. Tasks were synthetic; static cannot express the target; new draws repeat development; the weaker cross-time shuffle preserves readout and direction. In chemistry, the loop gave exact products for 76 of 200 held substrates, against 75 for nearest neighbour; the ARC-AGI-3 probe did not pass its pre-registered test.

8 What CORA cannot yet do

The failures fall into four kinds (Table 1), each stated as the measured result, what it taught us and the experiment it motivates: the thesis applied to the research itself. *Insufficient operation space.* On the public evaluation set (post hoc; exposed; not an official score), our neural comparison entry, with a contaminated

kind of failure	what we measured	what it taught us	what to change next
insufficient operation space	no typed hole of 425 failed source-half tasks matched one of the 211 operations; on 41 query outputs that only the neural entry answered correctly, ten times the budget found no program (post hoc, exposed)	the tested pieces did not match these holes	object-level construction, correct on 343 of 624 held query outputs; actions that draw on the background, edit object parts and assemble grids
inadequate representation	on 16 of our maize leaves a cell-image segmenter scored F1 near zero and CORA abstained on every leaf	the tested local edits did not rescue this representation	detect a collapsed baseline and change the representation first
ambiguous evidence	562 of 4,832 constructed rules that fit every demonstration were wrong on the query output (post hoc, development data)	fitting the demonstrations shows consistency, not the intended rule	choose observations where candidates disagree (45 to 63 of 217 in logic)
unreliable acceptance	a pre-registered calibrated acceptance rule, tested once on 100 held-out PUMA melanoma regions [60], accepted 605 edits, 430 harmful, lowering mean per-image F1 by 0.0020 (95% interval 0.0014 to 0.0027); the strict rule made no edit	it tested an acceptance rule, not learning from failure: its harm limit of 0.60, relaxed from 0.20 on the same calibration organs, did not hold on a new source (0.711)	tie any harm limit to the target's baseline accuracy, fixed before looking; abstain by default on a new source

Table 1: Four kinds of failure: what we measured, what it taught us, and what to change next; more rows are in Table 5.

measured limit (where it is measured)	next advance
a supplied operation space (0 of 425 failed tasks matched)	operations beyond the declared library, for example a learned proposer for typed holes whose proposals CORA checks
uncertain revision selection (no shown gain over best fixed order; 2 of 120 natural tasks)	learn when to change the representation, construct or search longer on natural tasks; a real-puzzle protocol has passed review as a draft
limited reuse (gain only on 1D-ARC)	learn when a kept operation applies; check it on another family first
ambiguity (562 of 4,832 fitting rules wrong)	choose informative observations (45 to 63 of 217 in logic)
noisy data (430 of 605 edits harmful)	calibrate acceptance on the target domain before accepting edits
no validated biological discovery	frozen predictions tested on independent measurements and interventions

Table 2: From measured limits to the next advances, ordered from problems already measured to harder, unsolved ones; none is claimed as solved.

checkpoint, was correct on 43 of the 172 query outputs in rehearsal and the CORA-only entry on 3; of the neural entry's 344 attempts, a CORA-CAST program reproducing every demonstration exists for 2, both correct and among CORA's own 3 answers, and for none of the 301 wrong ones; and on the 41 query outputs, in 36 tasks, that only the neural entry answered correctly, ten times the search budget found no program, with diagnostics suggesting missing actions as a language-coverage bottleneck, not proving that none exists (Table 9; rules refined on these tasks). Table 1's *unreliable acceptance* row has microscopy counterparts in Appendix E. Beyond these, the CORA-only entry solves few hidden-test puzzles (0.83%); the positive ARC-style results are on generated or previously scored related tasks, and we inspected the public evaluation set; the biology cohort was reused, its held-input vote was designed after a first result (Appendix F), and model-based results reconstruct known functions; no CORA hypothesis has been tested in a laboratory.

9 Toward learning which capability to construct

Open-endedness is the aim: a system that keeps finding what its language lacks, builds it and checks it on problems it was not built for. The experiments do not show that; they measure where the present system breaks, and Table 2 orders the next advances from limits already measured to harder ones. The next experiment, to be pre-registered, matches conditional construction against Popper [20, 21], an inductive logic programming learner that learns from failures and invents predicates, on the 595 sparse targets at equal observations and resources; it needs a validated whole-program four-input constraint inside Popper's search, not an output filter.

10 Conclusion

Reasoning systems can fail because a needed capability is unavailable, not merely because search was too short. CORA uses task-specific evidence to constrain, verify and retain candidate executable operations. On generated tasks, demonstrations constrained the missing operation after bounded search failed (64 against 16 with another task's holes, of 95); our failures on natural tasks led us to build object-level construction, correct

on 343 of 624 held, exposed outputs against 118 for static search (57 against 30 of 315 in two dimensions). Reuse helped on 1D-ARC only, and a fixed router improved on the pre-declared order of revisions, with no advantage established over the best post hoc order. This is a bounded step toward what the organisers describe as building symbolic models of a problem on the fly and testing explicit hypotheses [14, 15, 33, 36]: the space is supplied, and the scored entry, which checks but does not construct, scored 0.83%. Construction beyond a supplied space, revision selection on natural tasks and biological validation remain open. When reasoning fails, the failure is evidence; using that evidence well requires identifying what is missing and testing whether a proposed change actually addresses it.

Code and data. The scored notebooks (CORA-only, version 364571398; neural comparison, version 364496615), a verification notebook and the GitHub release (`github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI`; branch arc2026-sprint at commit 8ec11940, tag cora-arc2026-public-2026-11-08c) are public; the submitted version is tagged arc2026-paper-20261108.

Acknowledgements. Toni Kazic led high-level discussions on the big problems in science: what AI can and cannot do, grammar induction and program synthesis across domains, and what the next big leap could be. We thank Chris Fluharty, who started this project with us last year in a hackathon (the LLM route), and Pierce Taylor for high-level discussions on single-cell omics in biology and its bottlenecks (dispersion versus displacement), and for the joint single-cell work in the full paper.

References

[1] Tatsuya Akutsu, Satoru Miyano, and Satoru Kuhara. Identification of genetic networks from a small number of gene expression patterns under the Boolean network model. In *Pacific Symposium on Biocomputing*, volume 4, pages 17–28, 1999. doi: 10.1142/9789814447300_0003.

[2] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 942–963, 2025. URL `https://proceedings.mlr.press/v267/akyurek25a.html`. arXiv:2411.07279; first version titled "The Surprising Effectiveness of Test-Time Training for Abstract Reasoning".

[3] Simon Alford, Anshula Gandhi, Akshay Rangamani, Andrzej Banburski, Tony Wang, Sylee Dandekar, John Chin, Tomaso Poggio, and Peter Chin. Neural-guided, bidirectional program search for abstraction and reasoning. In Rosa Maria Benito, Chantal Cherifi, Hocine Cherifi, Esteban Moro, Luis M. Rocha, and Marta Sales-Pardo, editors, *Complex Networks & Their Applications X, Volume 1: Proceedings of the Tenth International Conference on Complex Networks and Their Applications COMPLEX NETWORKS 2021*, Studies in Computational Intelligence, pages 657–668. Springer, 2022. doi: 10.1007/978-3-030-93409-5_54. arXiv:2110.11536.

[4] Dana Angluin. Queries and concept learning. *Machine Learning*, 2:319–342, 1988. doi: 10.1023/A:1022821128753. URL `https://homepages.math.uic.edu/~lreyzin/papers/angluin88.pdf`.

[5] ARC Prize. Post on X, 16 October 2025. `https://x.com/arcprize/status/1978872651180577060`, 2025.

[6] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. URL `https://arxiv.org/abs/1611.01989`. arXiv:1611.01989.

[7] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Predictive inference with the jackknife+. *The Annals of Statistics*, 49(1):486–507, 2021. doi: 10.1214/20-AOS1965.

[8] Robert Cregar Berwick. Computational analogues of constraints on grammars: A model of syntactic acquisition. In *18th Annual Meeting of the Association for Computational Linguistics*, pages 49–53, Philadelphia, Pennsylvania, USA, 1980. Association for Computational Linguistics. doi: 10.3115/981436.981453. URL `https://aclanthology.org/P80-1014/`.

[9] Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages*, 7(POPL):1182–1213, 2023. doi: 10.1145/3571234. Article 41; arXiv:2211.16605.

[10] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html`. arXiv:2304.05128.

[11] François Chollet. On the measure of intelligence, 2019. URL https://arxiv.org/abs/1911.01547.
[12] François Chollet. Post on X, 21 August 2025. https://x.com/fchollet/status/1958410017683681698, 2025.
[13] François Chollet. Post on X, 6 August 2026. https://x.com/fchollet/status/2085324762637574183, 2026.
[14] François Chollet. Post on X, 14 August 2026. https://x.com/fchollet/status/2088243704603824311, 2026.
[15] François Chollet. Post on X, 21 August 2026. https://x.com/fchollet/status/2090838046937645398, 2026.
[16] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report, 2024. URL https://arxiv.org/abs/2412.04604.
[17] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems, 2025. URL https://arxiv.org/abs/2505.11831.
[18] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical report, 2026. URL https://arxiv.org/abs/2601.10904.
[19] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV 2000)*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169. Springer, 2000. doi: 10.1007/10722167_15.
[20] Andrew Cropper and Rolf Morel. Learning programs by learning from failures. *Machine Learning*, 110(4): 801–856, 2021. doi: 10.1007/s10994-020-05934-z. arXiv:2005.02259.
[21] Andrew Cropper and Rolf Morel. Predicate invention by learning from failures, 2021. URL https://arxiv.org/abs/2104.14426.
[22] James Cussens and Stephen Pulman. Incorporating linguistics constraints into inductive logic programming. In *Fourth Conference on Computational Natural Language Learning and the Second Learning Language in Logic Workshop*, pages 184–193, 2000. URL https://aclanthology.org/W00-0740/.
[23] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.
[24] Yu Feng, Ruben Martins, Osbert Bastani, and Isil Dillig. Program synthesis using conflict-driven learning. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 420–435. ACM, 2018. doi: 10.1145/3192366.3192382.
[25] John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 229–239. ACM, 2015. doi: 10.1145/2737924.2737977.
[26] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 17657–17671, 2025. URL https://proceedings.mlr.press/v267/franzen25a.html. arXiv:2505.07859.
[27] Gabriel Grand, Lionel Wong, Maddy Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. LILO: Learning interpretable libraries by compressing and documenting code. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.19791.
[28] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.
[29] Esther Heid, Daniel Probst, William H. Green, and Georg K. H. Madsen. EnzymeMap: curation, validation and data-driven prediction of enzymatic reactions. *Chemical Science*, 14(48):14229–14242, 2023. doi: 10.1039/d3sc02048g.
[30] Isaac Joffe and Chris Eliasmith. Vector symbolic algebras for the Abstraction and Reasoning Corpus. *npj Unconventional Computing*, 3(1):54, 2026. doi: 10.1038/s44335-026-00100-5. arXiv:2511.08747.
[31] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL https://arxiv.org/abs/2510.04871.
[32] Greg Kamradt. Announcing ARC-AGI-3: A new challenge for frontier agentic intelligence. ARC Prize blog, 2026. URL https://arcprize.org/blog/arc-agi-3-launch. Published 25 March 2026.
[33] Greg Kamradt. Post on X, 3 September 2026. https://x.com/GregKamradt/status/2095600045873828013, 2026.

[34] Subin Kim, Prin Phunyaphibarn, Donghyun Ahn, and Sundong Kim. Playgrounds for abstraction and reasoning. In *NeurIPS 2022 Workshop on Neuro Causal and Symbolic AI (nCSI)*, 2022. URL https://openreview.net/forum?id=F4RNpByoqP. Mini-ARC data: https://github.com/KSB21ST/MINI-ARC.

[35] Mike Knoop. ARC Prize 2025 results & analysis. ARC Prize blog, 2025. URL https://arcprize.org/blog/arc-prize-2025-results-analysis. Published 5 December 2025.

[36] Mike Knoop. Post on X, 16 July 2026. https://x.com/mikeknoop/status/2077819572452036896, 2026.

[37] Mike Knoop. Post on X, 3 September 2026. https://x.com/mikeknoop/status/2095600676919455857, 2026.

[38] Jan Krumsiek, Carsten Marr, Timm Schroeder, and Fabian J. Theis. Hierarchical differentiation of myeloid progenitors is encoded in the transcription factor network. *PLoS ONE*, 6(8):e22649, 2011. doi: 10.1371/journal.pone.0022649.

[39] John E. Laird. Introduction to Soar, 2022. URL https://arxiv.org/abs/2205.03854.

[40] John E. Laird, Paul S. Rosenbloom, and Allen Newell. Chunking in Soar: The anatomy of a general learning mechanism. *Machine Learning*, 1(1):11–46, 1986. doi: 10.1007/BF00116249.

[41] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[42] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/35ee921eb17e9b4c3e73b6e2ae0d55ba-Paper-Conference.pdf. arXiv:2411.02272.

[43] Isaac Liao and Albert Gu. ARC-AGI without pretraining, 2025. URL https://arxiv.org/abs/2512.06104.

[44] Amirreza Mahbod, Christine Polak, Katharina Feldmann, Rumsha Khan, Katharina Gelles, Georg Dorffner, Ramona Woitek, Sepideh Hatamikia, and Isabella Ellinger. NuInsSeg: A fully annotated dataset for nuclei instance segmentation in H&E-stained histological images. *Scientific Data*, 11:295, 2024. doi: 10.1038/s41597-024-03117-2.

[45] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 691–701, 2016. doi: 10.1145/2884781.2884807.

[46] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997. ISBN 0-07-042807-7. URL http://www.cs.cmu.edu/~tom/mlbook.html.

[47] Arseny Moskvichev, Victor Vikram Odouard, and Melanie Mitchell. The ConceptARC benchmark: Evaluating understanding and generalization in the ARC domain. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2305.07141.

[48] Stephen Muggleton and Wray Buntine. Machine invention of first-order predicates by inverting resolution. In John Laird, editor, *Proceedings of the Fifth International Conference on Machine Learning (ICML)*, pages 339–352, San Mateo, CA, 1988. Morgan Kaufmann. doi: 10.1016/B978-0-934613-64-4.50040-2.

[49] Chimdi Walter Ndubuisi. A phenotype atlas of maize lesion organization: geometry and a locked within-rater comparison. Unpublished working manuscript, 21 September 2026; aggregate results only, 2026.

[50] Chimdi Walter Ndubuisi. An image-derived segmentation pilot on maize leaf lesions. Unpublished exploratory report, November 2026; aggregate results only, 2026.

[51] Chimdi Walter Ndubuisi and Toni Kazic. Learning What to Change: Toward Failure-Grounded Co-Induction of Representations and Executable Reasoning Languages. Full technical report, University of Missouri, 2026. URL https://github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI/blob/arc2026-sprint/paper/v2_neural_cora/CORA_FULL_PAPER.pdf. Not peer reviewed.

[52] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 772–781, 2013. doi: 10.1109/ICSE.2013.6606623.

[53] Zhuofu Ni, Andrew E. Stine, Keith E. J. Tyo, and Linda J. Broadbelt. Curating a comprehensive set of enzymatic reaction rules for efficient novel biosynthetic pathway design. *Metabolic Engineering*, 65:79–87, 2021. doi: 10.1016/j.ymben.2021.02.006.

[54] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.09896. arXiv:2306.09896.

[55] Peter-Michael Osera and Steve Zdancewic. Type-and-example-directed program synthesis. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 619–630, 2015. doi: 10.1145/2737924.2738007.

[56] Oleksandr Polozov and Sumit Gulwani. FlashMeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 107–126, 2015. doi: 10.1145/2814270.2814310.

[57] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 49659–49688, 2025. URL https://proceedings.mlr.press/v267/pourcel25a.html. arXiv:2507.14172.

[58] Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991. doi: 10.1016/0004-3702(91)90015-C.

[59] Julio Saez-Rodriguez, Leonidas G. Alexopoulos, Jonathan Epperlein, Regina Samaga, Douglas A. Lauffenburger, Steffen Klamt, and Peter K. Sorger. Discrete logic modelling as a means to link protein signalling networks with functional analysis of mammalian signal transduction. *Molecular Systems Biology*, 5(1):331, 2009. doi: 10.1038/msb.2009.87.

[60] Mark Schuiveling, Hong Liu, Daniel Eek, Gerben E. Breimer, Karijn P. M. Suijkerbuijk, Willeke A. M. Blokx, and Mitko Veta. A novel dataset for nuclei and tissue segmentation in melanoma with baseline nuclei segmentation and tissue segmentation benchmarks. *GigaScience*, 14:giaf011, 2025. doi: 10.1093/gigascience/giaf011.

[61] Kensen Shi, Hanjun Dai, Kevin Ellis, and Charles Sutton. CrossBeam: Learning to search in bottom-up program synthesis. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2203.10452.

[62] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. Is the cure worse than the disease? Overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 532–543, 2015. doi: 10.1145/2786805.2786825.

[63] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15 (5–6):475–495, 2013. doi: 10.1007/s10009-012-0249-7.

[64] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.

[65] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025, 2025. URL https://github.com/1ytic/NVARC.

[66] Irene Stahl. The appropriateness of predicate invention as bias shift operation in ILP. *Machine Learning*, 20(1–2): 95–117, 1995. doi: 10.1007/BF00993476.

[67] Paul E. Utgoff. Shift of bias for inductive concept learning. In Ryszard S. Michalski, Jaime G. Carbonell, and Tom M. Mitchell, editors, *Machine Learning: An Artificial Intelligence Approach*, volume II, pages 107–148. Morgan Kaufmann, Los Altos, CA, 1986.

[68] Johan S. Wind. DSL solution to the ARC challenge. https://github.com/top-quarks/ARC-solution, 2020. First place, Kaggle Abstraction and Reasoning Challenge 2020.

[69] Yudong Xu, Wenhao Li, Pashootan Vaezipoor, Scott Sanner, and Elias B. Khalil. LLMs and the Abstraction and Reasoning Corpus: Successes, failures, and the importance of object-based representations. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2305.18354. Introduces 1D-ARC; data: https://github.com/khalil-research/1D-ARC.